Yggdrasil Platform — Ragnarok Data Migration Architecture

Mimir Labs Technical Publication

Document: ML-WP-008 **Version:** 1.0
Author: Christopher Gaither **Date:** March 2026
Classification: Public

0.1 Overview

Ragnarok is the deterministic execution layer of the Mimir Labs data platform. It is a standalone, air-gapped desktop application responsible for migrating data between enterprise systems using a structured, repeatable process.

Unlike traditional migration tooling, Ragnarok does not rely on AI, heuristics that change over time, or cloud services. All behavior is deterministic, inspectable, and reproducible. The system operates entirely locally, interacting only with source and target databases.

Ragnarok consumes structured manifests generated by Ratatosk and executes migrations against a target schema using the Mimisbrunnr semantic model as its reference framework.

0.2 Architectural Role

Ragnarok is the execution component within a three-part system:

- **Ratatosk:** discovery, classification, and manifest generation
- **Ragnarok:** mapping refinement, validation, and migration execution
- **Bifrost:** post-migration synchronization and orchestration

Ragnarok is not responsible for discovery or ongoing integration. Its role is singular: to take a defined semantic mapping and produce a controlled, auditable migration outcome.

0.3 Design Principles

Ragnarok is built around several non-negotiable constraints:

- **Determinism** — All classification, mapping, and transformation logic is rule-based and repeatable. Identical inputs produce identical outputs.
- **Air-Gapped Operation** — The system operates without internet connectivity. Data movement occurs only between explicitly defined source and target systems.
- **Target Preservation** — The target schema is treated as authoritative. No destructive or adaptive changes are made to the target. All transformations occur on the source side.
- **System Agnosticism** — Although optimized for Yggdrasil, Ragnarok can target any compatible PostgreSQL or SQL Server schema.

0.4 Migration Model

Ragnarok executes migrations through a staged pipeline. Each stage produces artifacts that feed the next, enforcing structure and traceability throughout the process.

0.4.1 Stage 1 — Taxonomy Classification

The source schema is analyzed and grouped into business domains. Classification is derived from:

- foreign key graph structure
- column archetypes
- naming patterns

This establishes the contextual boundary for all subsequent mapping.

0.4.2 Stage 2 — Automated Mapping

Source elements are matched to target structures using domain-constrained comparison techniques. Matching considers:

- string similarity
- synonym normalization
- domain alignment

This produces an initial mapping hypothesis.

0.4.3 Stage 3 — Semantic Review

Mappings are reviewed and corrected by an operator. This step ensures that semantic intent is preserved and prevents propagation of incorrect assumptions.

0.4.4 Stage 4 — Gap Analysis

The system identifies:

- unmapped source elements
- missing required target fields
- incompatible data types

This stage converts ambiguity into explicit decisions.

0.4.5 Stage 5 — Ingestion

Data is migrated using an ordered execution plan that respects relational dependencies and validation constraints.

0.5 Core Engines

0.5.1 Taxonomy Engine

Builds a structural understanding of the source system by combining relational topology with semantic inference. The output is a domain-scoped representation of the source schema.

0.5.2 Mapping Engine

Generates candidate mappings between source and target elements using constrained comparison techniques. Matching is limited to domain context to prevent cross-domain ambiguity.

0.5.3 Type Compatibility Layer

Normalizes data types across database platforms and evaluates transformation safety. Each mapping is classified as compatible, lossy, or invalid.

0.5.4 Migration Plan

The migration plan is the central execution artifact. It contains:

- table ordering based on dependency graph
- column mappings and transformations
- default value strategies
- validation rules

The plan is serializable, allowing pause, review, and resumption.

0.6 Execution Pipeline

Migration execution follows a strict sequence:

1. Load migration plan
2. Order tables via dependency graph
3. Extract data in batches
4. Apply transformations
5. Validate against constraints
6. Insert into target system
7. Resolve deferred relationships
8. Produce migration report

Bulk operations are optimized using database-native mechanisms to maximize throughput while maintaining validation guarantees.

0.7 Data Quality and Governance

Ragnarok treats migration as a governed process rather than a data transfer.

Validation includes:

- type conformity
- constraint enforcement
- referential integrity
- domain-specific rule checks

Metrics are captured throughout execution, including throughput, error rates, and coverage.

The system produces governance artifacts such as lineage reports, gap analyses, and migration manifests. These artifacts serve as inputs for downstream systems and audit processes.

0.8 Security Model

Ragnarok is designed for controlled environments where data sensitivity is high.

- No external network communication
- No persistent credential storage
- In-memory processing of sensitive data
- Full audit logging of migration activity

Access is restricted to authorized operators with administrative roles.

0.9 User Interface Model

The interface is structured as a linear workflow, ensuring that each stage is completed before proceeding. Key interaction surfaces include:

- schema exploration and classification
- mapping review and correction
- gap visualization
- real-time migration monitoring

The UI is not exploratory; it enforces process discipline.

0.10 Platform Significance

Ragnarok transforms data migration from an ad hoc engineering task into a controlled, semantic process. By separating classification, mapping, validation, and execution into discrete stages, it eliminates ambiguity and makes migration behavior predictable.

Within the broader platform, Ragnarok is the mechanism that converts semantic understanding into operational reality.

It does not define meaning. It enforces it.

Mimir Labs LLC | Harrisburg, PA